

PSI–FL Relation Table

Implementation-aligned algorithms for offline class alignment

This document distinguishes the conceptual exact-PSI baseline from the implemented fuzzy matching procedures in `label_mapping/rt_protocol.py`. Relation-table construction is an offline pre-processing phase before FL. The algorithms use no cross-client ground-truth labels for their decisions. The Python experiment simulates the parties centrally; local operations and messages below describe their intended deployment roles. A correct numerical similarity comparison does not guarantee semantic class identity: ambiguous classes may remain unmerged.

Implementation	Procedure in this document
<code>filter</code>	Description ranks (Algorithm 2), followed by the image-affinity intersection (Algorithm 5).
<code>affscan</code>	Image-affinity profiles directly supply fuzzy PSI (Algorithm 3).
<code>attn</code>	Keyword attention coordinates and confident mutual matching (Algorithm 4).
<code>attn_filter</code>	Default precision : text nomination, affinity ranks, full-feature verification, and consistent grouping (Algorithms 8 and 9).
Shared steps	Actual image affinity: Algorithm 6. Actual CKKS rank comparison: Algorithm 7.

Notation and eligibility. C_i denotes client i ’s eligible classes: a local summary exists and the sample count meets the public minimum. Ineligible classes still receive singleton global ids. π_i is the client’s private permutation from local ids to masked ids; all transmitted pair indices are masked ids. Matrices below are ordered by masked id. Local real-id notation is retained only for readability. $\Delta_T, \Delta_A, \Delta_V$ are publicly specified ladders for text, affinity-profile, and full-feature verification similarities. The MNIST harness uses

$$\Delta_T = \{.90, .85, \dots, .40\}, \quad \Delta_A = \{.990, .985, \dots, .950\},$$

$$\Delta_V = \{.50, .55, \dots, .85\} \cup \{.900, .902, \dots, .998\}.$$

These are implementation settings, not universal accuracy guarantees. Other callers can configure the ladders. An allowed pair receives rank

$$R[a, b] = \sum_{\delta \in \Delta} \mathbf{1}\{s_i[a]^\top s_j[b] > \delta\}.$$

Thus δ is a *similarity threshold*, not a distance tolerance. The strict comparison agrees with the implementation.

Shared subroutines

```

1: procedure UNIT( $V$ )
2:   for each row  $v$  of  $V$  do
3:      $v \leftarrow v / \max(\|v\|_2, 10^{-12})$ 
4:   end for
5:   return  $V$ 
6: end procedure

7: procedure CENTERUNIT( $S$ )
8:   return UNIT( $S - \text{rowmean}(S)$ )
9: end procedure

10: procedure ONESIDED( $R$ )
11:    $M \leftarrow \emptyset$ 
12:   for each row  $a$  do
13:      $m \leftarrow \max_b R[a, b]$ 
14:     if  $m > 0$  and the maximizing column  $b$  is unique then
15:        $M \leftarrow M \cup \{(a, b)\}$ 
16:     end if
17:   end for
18:   return  $M$  // zero rows and exact ties abstain
19: end procedure

20: procedure MUTUAL( $R$ )
21:   return  $\{(a, b) \in \text{ONESIDED}(R) : (b, a) \in \text{ONESIDED}(R^\top)\}$ 
22: end procedure

23: procedure RANKBOUNDS( $R, \Delta$ )
24:   sort thresholds as  $d_1 \leq \dots \leq d_m$ ; set  $d_0 = -1, d_{m+1} = 1$ 
25:   for each pair  $(a, b)$  with rank  $r = R[a, b]$  do
26:      $L[a, b] \leftarrow d_r; U[a, b] \leftarrow d_{r+1}$ 
27:   end for
28:   return  $(L, U)$ 
29: end procedure

```

Normalization and bounds. Unit normalization removes magnitude; unrelated embedding spaces still need alignment. For evaluated, unmasked pairs, RANKBOUNDS encloses the plaintext similarity when threshold bits are exact. CKKS approximation can change bits near a threshold, so the bounds are not unconditional certificates for noisy decryptions. Experimental diagnostics check plaintext/HE disagreements but do not determine matches. A mask-forced zero rank provides no bound and is excluded before acceptance.

Code mapping. These helpers implement `unit`, `center_unit`, `one_sided`, `mutual`, and `rank_bounds`. Neither tie rejection nor mutual matching guarantees precision.

Experiment group table. After client-level alignment, `to_group_map` applies majority voting within a known common party/local-label schema, enforces injective party assignments, and rennumbers ids. This training adapter supplies no additional cross-client semantic evidence.

Algorithm 1 Shared-vocabulary description PSI

STATUS. Conceptual exact-PSI baseline; not implemented by `run_rt_protocol`.

ASSUMPTION. There exists a shared vocabulary W : a published list of keywords/phrases that every client draws on when describing what a class *is*. Every client's description of the same real-world concept is therefore built from the same word pool, so identical concepts tend to yield identical or overlapping description strings.

OBJECTS

C_i client i 's local set of classes (real ids).

π_i client i 's private permutation $\text{id} \mapsto \text{masked_id}$. Only masked ids ever leave client i .

W the shared public vocabulary.

$\text{desc}_i[a] \subseteq W$ the keyword(s) client i attaches to class a , describing it.

PHASE 1 — build the local table. Local only; no communication.

```
1: for each client  $i$  do
2:   for each class  $a \in C_i$  do
3:      $\text{masked\_id} \leftarrow \pi_i(a)$ 
4:     choose  $\text{desc}_i[a] \subseteq W$ : the keywords that describe class  $a$ 
5:      $\text{Table}_i \leftarrow \text{Table}_i \cup \{(a, \text{masked\_id}, \text{desc}_i[a])\}$  // the local |id|masked_id|description| table
6:   end for
7: end for
```

PHASE 2 — PSI on descriptions. Between each client pair; no aggregator.

```
8: for each client pair  $(i, j)$ ,  $i < j$  do
9:    $E_{ij} \leftarrow \emptyset$ 
10:  run EXACTPSI between  $\{\text{desc}_i[a] : a \in C_i\}$  and  $\{\text{desc}_j[b] : b \in C_j\}$  // plain PSI: reveals only which
    description strings are shared, not the descriptions of the non-matching classes
11:  for each matched description  $d$ , held by class  $a$  at  $i$  and class  $b$  at  $j$  do
12:     $E_{ij} \leftarrow E_{ij} \cup \{(\pi_i(a), \pi_j(b))\}$  // a masked-id-to-masked-id relation – never real ids
13:  end for
14: end for
15:  $E \leftarrow \bigcup_{i < j} \{(i, \hat{a}, j, \hat{b}, 1) : (\hat{a}, \hat{b}) \in E_{ij}\}$ 
16: return  $E$  // this is what is handed to the aggregation server – see Algorithm 9
```

Algorithm 2 Fuzzy description PSI followed by the original image-affinity filter

Require: eligible classes C_i , descriptions $d_i[a]$, aligned text encoders, Δ_T, Δ_A

Ensure: weighted masked-id candidate relations E

Local preparation.

```
1: for each client  $i$  do
2:   for each  $a \in C_i$  do
3:     choose description  $d_i[a]$  and compute its embedding  $t_i[a]$ 
4:     store  $(a, \pi_i(a), d_i[a])$  locally
5:   end for
6:    $t_i \leftarrow \text{UNIT}(t_i)$ 
7:    $(s_i, v_i) \leftarrow \text{IMAGE SIGNALS}(i)$ 
8: end for
Pairwise matching.
9:  $E \leftarrow \emptyset$ 
10: for each client pair  $i < j$  do
11:    $R_T \leftarrow \text{FPSIRANK}(t_i, t_j, \Delta_T, \mathbf{1})$ 
12:    $E'_{ij} \leftarrow \text{MUTUAL}(R_T)$ 
13:    $(E_{ij}, R_A) \leftarrow \text{IMAGEAFFINITYFILTER}(E'_{ij}, s_i, s_j, \Delta_A)$ 
14:   add  $(i, \pi_i(a), j, \pi_j(b), R_T[a, b] + R_A[a, b])$  to  $E$  for  $(a, b) \in E_{ij}$ 
15: end for
16: return  $E$  to Algorithm 9
```

// Algorithm 6

IMPLEMENTATION. `method="filter"`. The code uses supplied `desc_vecs`, or character 1–3-gram hashing if none are supplied. The MNIST semantic baseline averages individually encoded description keywords before normalization. Unlike anchor-indexed attention, this direct-description baseline requires aligned encoder coordinates for a meaningful dot product.

Algorithm 3 Affinity-generated descriptions: image profiles directly drive fuzzy PSI

Require: eligible local image sets X_i^a , common public image encoder F , ordered public probes P , Δ_A

Ensure: weighted masked-id relations E

Local preparation.

```
1: for each client  $i$  do
2:    $(s_i, v_i) \leftarrow \text{IMAGE SIGNALS}(i)$ 
3:   for each  $a \in C_i$  do
4:     store  $(a, \pi_i(a), s_i[a])$  locally
5:   end for
6: end for
Pairwise matching.
7:  $E \leftarrow \emptyset$ 
8: for each client pair  $i < j$  do
9:    $R_A \leftarrow \text{FPSIRANK}(s_i, s_j, \Delta_A, \mathbf{1})$ 
10:   $M_{ij} \leftarrow \text{MUTUAL}(R_A)$ 
11:  add  $(i, \pi_i(a), j, \pi_j(b), R_A[a, b])$  to  $E$  for  $(a, b) \in M_{ij}$ 
12: end for
13: return  $E$  to Algorithm 9
```

// actual cosine construction: Algorithm 6

IMPLEMENTATION. `method="affscan"`. The fuzzy-PSI inputs are normalized *affinity-profile vectors*, not raw images, names, or individual probe labels. Text is unused. This is a visual-only baseline; Algorithm 4 instead derives its queries from class descriptions.

Algorithm 4 Keyword attention coordinates used by the implementation

Require: ordered public text anchors A , client-local text encoder E_i , keyword lists $D_i[a]$, $\tau > 0$

Ensure: text coordinates $z_i[a]$ indexed by the public anchor list

```
1: for each client  $i$  do
2:    $K_i \leftarrow \text{UNIT}([E_i(w)]_{w \in A}); n \leftarrow |C_i|$ 
3:    $K_{c,i} \leftarrow K_i - \text{columnmean}(K_i)$ 
4:   compute compact SVD  $K_{c,i} = U\Sigma V^\top$ 
5:   retain singular values  $\sigma > \max(10^{-6}\sigma_{\max}, 10^{-12})$ 
6:    $W_i \leftarrow U_{\text{keep}}\Sigma_{\text{keep}}^{-1}U_{\text{keep}}^\top$ 
7:    $\text{df}_i(w) \leftarrow |\{a \in C_i : w \in D_i[a]\}|$ 
8:   for each class  $a \in C_i$  do
9:     encode every keyword separately:  $q_k \leftarrow \text{UNIT}(E_i(D_i[a]_k))$ 
10:     $c_a(w) \leftarrow$  multiplicity of keyword  $w$  in  $D_i[a]$ 
11:     $\tilde{w}_k \leftarrow \log(n/\text{df}_i(D_i[a]_k))/c_a(D_i[a]_k)$ 
12:    if all  $\tilde{w}_k = 0$  then
13:       $\tilde{w}_k \leftarrow 1/c_a(D_i[a]_k)$  for every keyword
14:    end if
15:     $w_k \leftarrow \tilde{w}_k / \sum_\ell \tilde{w}_\ell$ 
16:     $\alpha_k \leftarrow \text{softmax}(q_k K_i^\top / \tau)$  for each keyword
17:     $\bar{\alpha}_i[a] \leftarrow \sum_k w_k \alpha_k$ 
18:     $\ell_i[a] \leftarrow \log(\max(\bar{\alpha}_i[a], \varepsilon_{\text{tiny}}))$  elementwise
19:     $z_i[a] \leftarrow \text{UNIT}((\ell_i[a] - \text{mean}(\ell_i[a]))W_i)$ 
20:  end for
21: end for
Standalone attn matching.
22: for each pair  $i < j$  do
23:    $R_T \leftarrow \text{FPSIRANK}(z_i, z_j, \Delta_T, \mathbf{1})$ 
24:    $R_T[a, b] \leftarrow 0$  wherever  $R_T[a, b] < |\Delta_T|$ 
25:   emit masked relations from  $\text{MUTUAL}(R_T)$ , weighted by  $R_T[a, b]$ 
26: end for
```

IMPLEMENTATION. `keyword_attn` and `anchor_log_coordinates`, default `coord="log_whiten"`, one head. With legacy `desc_vecs`, a whole description is one query and receives weight 1; without keyword metadata, weights are uniform. No projection head or keyword weights are trained. Different text encoders embed the same anchor *list*; identical numerical keys are not required. This shared indexing does not guarantee semantic alignment of arbitrary encoders. With one active keyword, the corrected coordinate is an anchor-span projection.

Algorithm 5 Image-affinity intersection filter: original cascade

Require: text candidate pairs E'_{ij} ; full affinity profiles s_i, s_j from Algorithm 6; Δ_A

Ensure: surviving pairs and their affinity ranks

- 1: **procedure** IMAGEAFFINITYFILTER($E'_{ij}, s_i, s_j, \Delta_A$)
- 2: $I \leftarrow \{a : (a, b) \in E'_{ij}\}; J \leftarrow \{b : (a, b) \in E'_{ij}\}$
- 3: $M[a, b] \leftarrow \mathbf{1}\{a \in I \text{ and } b \in J\}$
- 4: $R_A \leftarrow \text{FPSIRANK}(s_i, s_j, \Delta_A, M)$
- 5: **return** $(E'_{ij} \cap \text{MUTUAL}(R_A), R_A)$
- 6: **end procedure**

IMPLEMENTATION. This is the filter stage of `method="filter"`, and the legacy `method="attn_filter"` with `attn_match="intersection"` path. The mask covers the Cartesian product of candidate endpoint sets, not only the original candidate edges. Mutual matching is evaluated in that masked matrix. Pairs rejected by the earlier text stage cannot be recovered by this cascade. The current precision-mode replacement is Algorithm 8.

ACTUAL AFFINITY. The implemented signal is the cosine profile from Algorithm 6. It is not an unspecified classifier score, k NN fraction, posterior, or affinity learned from the other client's labels.

Algorithm 6 Actual image-affinity construction supplied to fuzzy PSI

Require: common frozen image encoder F , ordered public unlabeled probes $P = \{p_1, \dots, p_m\}$

Require: client i 's private selected samples X_i^a for each eligible class a

Ensure: normalized affinity profiles $s_i[a]$ and full-feature vectors $v_i[a]$

```
1: procedure IMAGESIGNALS( $i$ )
2:   compute public features  $f_p \leftarrow F(p)$  for each  $p \in P$ 
3:    $\mu \leftarrow m^{-1} \sum_{p \in P} f_p$ ;  $h_p \leftarrow f_p - \mu$ 
4:   for each  $a \in C_i$  do
5:      $r_i(a) \leftarrow |X_i^a|^{-1} \sum_{x \in X_i^a} F(x) - \mu$  // accumulate in batches locally
6:      $v_i[a] \leftarrow r_i(a) / \max(\|r_i(a)\|_2, 10^{-12})$ 
7:     for each public probe  $p$  do
8:        $S_i[a, p] \leftarrow v_i[a]^\top h_p / \max(\|h_p\|_2, 10^{-12})$  // affi(a, p)
9:     end for
10:     $s_i[a] \leftarrow \text{UNIT}(S_i[a] - m^{-1} \sum_{p \in P} S_i[a, p])$ 
11:  end for
12:  return ( $s_i, v_i$ )
13: end procedure
```

TWO DISTINCT SIMILARITIES.

Local probe affinity: $\text{aff}_i(a, p) = \cos(r_i(a), h_p)$.

Cross-client profile similarity tested by fuzzy PSI:

$$c_A(a, b) = s_i[a]^\top s_j[b].$$

Additional precision-mode verification similarity:

$$c_V(a, b) = v_i[a]^\top v_j[b].$$

The protocol does not compare a class directly with another client's raw images. Public probe labels are never used. The two visual signals share the same encoder and samples; they are complementary representations, not statistically independent evidence.

IMPLEMENTATION. `aff_profile(R, P)` computes $\text{UNIT}(R)\text{UNIT}(P)^\top$ on already public-mean-centered features; `center_unit` then normalizes its rows. The MNIST harness uses a shared pretrained ResNet-18 with the classifier head removed and consistent image preprocessing. Legacy summaries use at most 64 samples/class; precision mode uses `precision_summ` from all available samples, falling back to the supplied `summ` if absent. The server setting `rt_exemplars_per_class=0` also selects all available samples.

Algorithm 7 Implemented CKKS labeled fuzzy-PSI rank comparison

Require: normalized rows $s_i[a], s_j[b]$ in a compatible coordinate space, ladder Δ , allowed mask M

Ensure: masked pairwise rank matrix R , subsequently used by MUTUAL

```
1: procedure FPSIRANK( $s_i, s_j, \Delta, M$ )
2:   receiver  $j$  generates a CKKS context and retains its secret key
3:    $j$  sends the public context and encrypted, column-packed coordinates of  $s_j$ 
4:    $R \leftarrow 0$ 
5:   for each sender-row block and each pair  $(a, b)$  represented by its slots do
6:     sender  $i$  evaluates  $\hat{c}_{ab} \leftarrow \sum_{k=1}^d s_i[a, k] \text{Enc}_j(s_j[b, k])$ 
7:     for each  $\delta \in \Delta$  do
8:        $r_{ab\delta} \leftarrow \text{Uniform}(1, 1000)$  independently
9:        $\hat{u}_{ab\delta} \leftarrow (\hat{c}_{ab} - \delta) r_{ab\delta} M[a, b]$ 
10:       $i$  sends packed  $\hat{u}_{ab\delta}$  to receiver  $j$ 
11:       $j$  decrypts  $u_{ab\delta}$  and sets  $B_\delta[a, b] \leftarrow M[a, b] \mathbf{1}\{u_{ab\delta} > 0\}$ 
12:       $R[a, b] \leftarrow R[a, b] + B_\delta[a, b]$ 
13:    end for
14:  end for
15:   $j$  returns  $R$  to  $i$ ; return  $R$ 
16: end procedure
```

PLAINTEXT REFERENCE. $\text{FPSIRANK}(s_i, s_j, \Delta, M) = M \odot \sum_{\delta \in \Delta} \mathbf{1}\{s_i s_j^\top > \delta\}$. This is `fpsi_rank_plain`; it is a debugging/reference mode, not private PSI.

PACKING. `FPSIRReceiver` uses polynomial modulus degree 8192 (4096 slots), coefficient-modulus bits [60, 40, 40, 60], and scale 2^{40} . For n_j receiver classes, each coordinate column repeats $\lfloor 4096/n_j \rfloor$ times. Sender row blocks repeat their coordinate over each receiver class. The loop above is written slotwise for clarity; encrypted products are batched.

LEAKAGE AND NUMERICS. The receiver sees masked decrypted magnitudes u , not merely match bits. The parties also learn ranks, class counts, and the masking/selection information needed for the next stage. Finite-range multipliers do not constitute a proof of ideal “bits-only” leakage. CKKS is approximate and can disagree with the strict plaintext comparison near a threshold. Finer ladders disclose finer similarity ranges and increase computation/communication. These limitations apply to the implemented semi-honest prototype; no exact cryptographic PSI guarantee is claimed.

CALLERS. Algorithms 2–6 supply text or affinity-coordinate vectors. Algorithm 8 additionally supplies full image-feature vectors. Only the vectors and ladder change; the encrypted threshold mechanism is the same.

Algorithm 8 Precision-first attention-affinity preprocessing: current `psi_attn_filter`

Require: z_i from Algorithm 4, (s_i, v_i) from Algorithm 6

Require: ladders $\Delta_T, \Delta_A, \Delta_V$; verification floor $\eta = .90$, margin $\gamma = .01$

Ensure: weighted edges E , supported pair set H , uncertain candidates for local review

```
1:  $E, H \leftarrow \emptyset$ 
2: for each client pair  $i < j$  do
3:    $R_T \leftarrow \text{FPSIRANK}(z_i, z_j, \Delta_T, \mathbf{1})$ 
4:    $M[a, b] \leftarrow \mathbf{1}\{R_T[a, b] > 0\}$ 
5:    $R_A \leftarrow \text{FPSIRANK}(s_i, s_j, \Delta_A, M)$  // zero if no allowed pairs
6:    $R_V \leftarrow \text{FPSIRANK}(v_i, v_j, \Delta_V, \mathbf{1})$  // all competitors, unmasked
7:    $(L_T, U_T) \leftarrow \text{RANKBOUNDS}(R_T, \Delta_T)$ 
8:    $(L_A, U_A) \leftarrow \text{RANKBOUNDS}(R_A, \Delta_A)$ ;  $(L_V, U_V) \leftarrow \text{RANKBOUNDS}(R_V, \Delta_V)$ 
9:    $B \leftarrow 0$ ;  $G \leftarrow 0$ 
10:  for each pair  $(a, b)$  do
11:     $T \leftarrow [L_T[a, b] \geq \max \Delta_T]$ ;  $I \leftarrow [L_A[a, b] \geq \max \Delta_A]$ 
12:    if  $R_T[a, b] = 0$  or  $R_A[a, b] = 0$  or  $\neg(T \vee I)$  or  $L_V[a, b] < \eta$  then
13:      continue
14:    end if
15:     $q \leftarrow \max(\{U_V[a, b'] : b' \neq b\} \cup \{U_V[a', b] : a' \neq a\} \cup \{-1\})$ 
16:    if  $(T \wedge L_V[a, b] \geq q - \gamma)$  or  $(\neg T \wedge L_V[a, b] \geq q + \gamma)$  then
17:       $B[a, b] \leftarrow 1$ ;  $G[a, b] \leftarrow (L_T[a, b] + L_A[a, b] + L_V[a, b])/3$ 
18:      add the masked unordered pair  $\{(i, \pi_i(a)), (j, \pi_j(b))\}$  to  $H$ 
19:    end if
20:  end for
21:  add  $(i, \pi_i(a), j, \pi_j(b), G[a, b])$  to  $E$  for  $(a, b) \in \text{MUTUAL}(G)$ 
22:  record strongly nominated pairs failing verification as uncertain, not as confirmed errors
23: end for
24: return  $(E, H)$  to Algorithm 9; retain review records locally
```

IMPLEMENTATION. `attn_match="precision"`, `verified_candidates`, and `rank_bounds`. Confidence and margins use only threshold ranks. R_V must include every image-feature rival: hiding rivals before checking the margin could create artificial confidence. Strong text may resolve visually close alternatives within γ ; weak-text image rescue instead requires a margin of γ over all rivals. The image-feature floor applies in both cases.

PREVIOUS FUSION ABLATION. With `attn_match="fusion"`, there is no R_V or complete-link check. Positive text and affinity ranks, with at least one highest-rung rank, receive score $R_T|\Delta_A| + R_A|\Delta_T|$, then undergo mutual matching. Precision mode is a deliberate extension of that earlier procedure.

Algorithm 9 Global relation table with optional complete-link verification; subsequent FL queries

Require: weighted masked edges E , every client's full masked-id list

Require: optional supported-pair set H from Algorithm 8

```
1: initialize each  $(i, \hat{a})$  as a singleton union-find component
2: for each edge  $(i, \hat{a}, j, \hat{b}, w)$  in decreasing order of  $w$  do
3:   let  $A, B$  be the endpoint components
4:   if  $A = B$  then
5:     continue
6:   end if
7:   if  $A$  and  $B$  contain ids from the same client then
8:     reject this merge
9:   else if  $H$  is supplied and some  $u \in A, v \in B$  satisfy  $\{u, v\} \notin H$  then
10:    reject this merge and record the consistency conflict
11:  else
12:    merge  $A$  and  $B$ 
13:  end if
14: end for
15: assign one global id to every resulting component, including singletons
16: return GlobalTable = |global_id|client_id|masked_id|
```

FEDERATED LEARNING: reuse the completed table.

```
17: for each global class  $g$  used in a training round do
18:   for each  $(i, \hat{a})$  assigned to  $g$  do
19:     aggregator queries client  $i$  with masked id  $\hat{a}$ 
20:     client  $i$  resolves its real id locally as  $a = \pi_i^{-1}(\hat{a})$ 
21:     client returns the authorized class update/statistic tagged with  $\hat{a}$ 
22:   end for
23:   aggregator combines the updates for  $g$ 
24: end for
```

IMPLEMENTATION. global_table. With H , a transitive path alone is insufficient: every cross-component pair needs support. This can reduce recall under noisy or sparse observations. The precision path removes rejected edges from its returned edge list and exports review records. A singleton may be a genuinely private class or an unresolved shared class.

DISCLOSURE. Enforcing the complete-link rule requires the aggregator to access the supported masked-pair relation H in addition to selected edges. The simulation passes this relation directly; a deployment must account for this additional disclosed relation, as well as the extra full-feature ranks from Algorithm 8.
